

CNN from Scratch for MNIST Digit Classification

Introduction and Task Description

The main goal of this project is to create a Convolutional Neural Network (CNN) from scratch and use it to classify handwritten digits from the MNIST dataset without using any frameworks for the model itself like PyTorch or TensorFlow. I will use torchvision to load the MNIST data and I will then implement it manually using NumPy

I choose MNIST since it is a good dataset to show the key notions behind CNNs. . Each image is a 28×28 grayscale image of a digit between 0 and 9, so the task is a ten-class image classification problem. Convolutional neural networks are particularly suited to this type of challenge, as they can learn local visual properties such as edges, curves and forms while keeping the spatial structure of the image (Goodfellow, Bengio and Courville, 2016; Stanford University, 2025).

The model consists of convolution layers, relu activation, max pooling, flattening, fully connected layer, softmax classification and cross-entropy loss. The purpose of doing this project manually is to understand the working of the network from inside and not by taking CNN as a black box.

CNN Background and Convolution Theory

The main idea behind a CNN is convolution. Convolution layers utilise a small matrix called a kernel or filter, that slides across an image and takes a weighted sum over small neighbourhoods of pixels. This results in a feature map, which shows us where certain patterns have been found in the image. This is useful for image data since the network can learn local patterns without immediately flattening the image and losing spatial information (Goodfellow, Bengio and Courville, 2016).

Mathematically, if the input is X , the kernel weights are W , the bias is b , and the output feature map is Y , the convolution operation can be written as:

$$Y_{c_{\text{out}},i,j} = b_{c_{\text{out}}} + \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} W_{c_{\text{out}},c,u,v} X_{c,i+u,j+v}$$

Here, c_{out} is the output channel, C_{in} is the number of input channels, and k_H and k_W are the height and width of the kernel. In simple words, the filter looks at a small region of

the image, multiplies the pixel values by the filter weights, adds them together, and stores the result in the feature map.

After convolution, I then apply the ReLU activation function: $f(x) = \max(0, x)$ as it introduces non-linearity, which is important because without non-linear activation functions, adding more layers would not significantly increase what the network can learn.

I also implement max pooling, as max pooling reduces the size of the feature maps by keeping the largest value from each small window. This helps decrease computation while keeping the strongest identified features. The output size after a convolution layer is calculated using:

$$\text{Output size} = \frac{W - K + 2P}{S} + 1$$

where W is the input size, K is the kernel size, P is padding, and S is stride.

Architecture and Code Implementation

I have load the MNIST dataset. The MNIST data contains 70,000 images of handwritten digits with 10 class labels, and each image is stored as 28×28 grayscale pixels (UCI Machine Learning Repository, 2021). Each image has one channel, before being fed into the CNN, each image has shape: $1 \times 28 \times 28$

The pixel values is normalised by dividing by 255, so, it lie between 0 and 1. Making the input values smaller, the training process will become more stable.

The first convolution layer has 8 filters of size 3×3 , with stride 1 and no padding. So using the output size formula I get : $\frac{28-3+2(0)}{1} + 1 = 26$. So, the first convolution layer results in: $8 \times 26 \times 26$

The code, uses `im2col` to implement convolution. Rather than performing the convolution with just slow nested loops, `im2col` rearranges image patches from the image into columns so that the convolution can be performed using matrix multiplication: $Y_{\text{flat}} = W_{\text{row}} X_{\text{col}} + b$

Then, ReLU is applied, after the first convolution and a 2×2 max pooling layer with stride 2 is applied. This decreases the output from: $8 \times 26 \times 26$ to : $8 \times 13 \times 13$.

The second convolution layer has 16 filters of size 3×3 . Now the input size is now 13×13 , so the output size is: $\frac{13-3+2(0)}{1} + 1 = 11$. This gives: $16 \times 11 \times 11$

Another ReLU activation is applied and another 2×2 max pooling layer. Reducing the output to: $16 \times 5 \times 5$. The output is then flattened: $16 \times 5 \times 5 = 400$

These 400 learned features are passed into a fully connected layer, which outputs 10 output values, one for each digit class. The fully connected layer is: $z = Wx + b$

Finally, the softmax function converts the 10 output values into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

The digit with the highest probability is selected as the model's prediction.

Training Strategy

The model was trained using mini-batch stochastic gradient descent. In the notebook, the training settings were: $N_{STEPS} = 3000$, $BATCH = 8$, $lr = 0.01$

This means the model was trained for 3000 update steps, using 8 images in each mini-batch, with a learning rate of 0.01. Optimisation methods are particularly suited to mini-batch stochastic gradient descent, where a randomly selected mini-batch is utilised to update the parameters instead of using the whole dataset at each step. (Hanada, 2026).

The loss function used was cross-entropy loss: $L = -\sum_{i=1}^{10} y_i \log(p_i)$

This is useful because MNIST classification is a multi-class classification problem with 10 possible classes. The true label is represented as a one-hot vector, and I wanted the model to increase the probability of the correct digit.

The parameters are updated using gradient descent: $\theta \leftarrow \theta - \alpha \nabla_{\theta} L$

where θ is the model parameters, α is the learning rate, and $\nabla_{\theta} L$ is the gradient of the loss. The notebook also used He/Kaiming initialisation for the weights:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

This type of initialisation is good for networks using ReLU activation functions, as it helps keep the scale of activations more stable during training (He et al., 2015).

Backpropagation and Gradient Checking

Backpropagation is implemented manually in the notebook so that the model could update its own weights during training. After the forward pass calculates the prediction and loss, the gradients are then passed backwards through the network.

For softmax with cross-entropy loss, the gradient becomes: $\frac{\partial L}{\partial z} = p - y$

where p is the predicted probability vector and y is the true one-hot encoded label.

For the fully connected layer, $z = Wx + b$

the gradients are: $\frac{\partial L}{\partial W} = \delta x^T$, $\frac{\partial L}{\partial b} = \delta$, $\frac{\partial L}{\partial x} = W^T \delta$

These gradients are then used to update the weights and biases, while also passing the error back to the earlier layers.

For the convolution layer, I used the same im2col structure from the forward pass. Since convolution was written as a matrix multiplication, the backward pass could be handled in a similar way to the fully connected layer. The col2im function was then used to reshape the gradients back into the original image or feature-map shape.

Before training, I also used numerical gradient checking to check that my

backpropagation code was working correctly: $\frac{\partial L}{\partial \theta} \approx \frac{L(\theta+h) - L(\theta-h)}{2h}$

This compared the manually calculated gradients with a numerical approximation. A small relative error showed that the backward pass was implemented correctly.

Results and Performance

After training, I tested the model on the MNIST test set to see how well it performed on unseen images.

The accuracy was calculated as: $\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of test images}} \times 100$

In my last run, the model got a test accuracy of 95.23% overall. This is a good result for a CNN built from scratch with NumPy, especially because the model is fairly small and does not use a deep learning framework for the actual implementation.

I also looked at the accuracy on each digit to get a better understanding of the performance. The model performed best on digits 0, 1 and 2, with accuracies of 99.4%, 98.3% and 97.1%. The worst results were for digits 8 and 9 with accuracies of 91.8% and 91.9%. This shows that the model did well overall but some digits were still harder to classify than others.

I also plotted the training loss curve to check if the model was learning during the training. As the loss decreased over time, it showed that the model was getting better at making predictions and updating the parameters in the correct direction.

Finally, I examined the predictions in more detail using a confusion matrix. This helped identify which digits were correctly classified and which digits were confused with each other, giving a clearer picture of the model's performance than accuracy alone could give.

The performance analysis was supported by the training loss curve and confusion matrix shown in the notebook.

Analysis of Results

These results indicate that the CNN managed to learn useful patterns from the MNIST images, even though the model was built from scratch using NumPy. This is a good result as we kept the network quite simple, with only two convolution layers and one fully connected layer.

The per-class results give a better sense of the model's behaviour across the different digits. The best results were for the 0, 1 and 2 digits with accuracies of 99.4%, 98.3% and 97.1%. Probably the model finds it easier to classify these digits because they have more distinctive shapes. For digits 8 and 9 the results were weaker with accuracies of 91.8% and 91.9%. This indicates that some digits are more difficult to distinguish, particularly when the style of handwriting causes them to resemble other classes.

The confusion matrix also helped to show where the model had made mistakes. For example, the digits 4 and 9, or 3 and 5, can sometimes look very similar visually depending on how they are written. This explains why the model sometimes confuses certain classes even though the overall accuracy is high.

Overall the results are reasonable for a manually implemented CNN from scratch. They show that the main parts of the model including convolution, ReLU, max pooling, flattening and softmax classification are working together as expected. However, since the model is implemented in NumPy and trained on the CPU, it is slower and less optimised than a CNN built using a dedicated deep learning framework.

Limitations and Improvements

One of the main limitations of this project is that the CNN was kept somewhat minimal, with two convolution layers and one fully connected layer. This made it easier to generate from scratch but also set limits on the complexity of what the model could learn.

The training procedure was easy as well. I chose mini-batch stochastic gradient descent because it fits nicely with the optimisation techniques discussed in the module, but the final result is still sensitive to parameters such as the learning rate, batch size and number of training steps.

Another con is speed. The model is implemented in NumPy and trained on the CPU, which means it is slower than a CNN implemented with an efficient deep learning framework. The im2col approach did improve the efficiency of the convolution, but it is still not as fast as GPU-based solutions.

To improve the model I can train it for a greater number of steps, change the learning rate, change the batch size, add batch normalisation or add more filters. In the lectures, batch normalisation is presented as a technique to deal with issues that arise from the changing distributions of inputs in hidden layers during training (Hanada, 2026). Another

way could be to test the same idea on CIFAR-10 but this would be harder as the images are in colours and more complicated than MNIST.

Conclusion

Overall, this project gave me a better understanding of the mechanics of CNNs and how they work outside of just using a deep learning library. I built the whole CNN pipeline myself: loading and normalising the data, convolution, ReLU, pooling, softmax, cross-entropy loss, and backpropagation in order to see how an image is gradually transformed into useful features for classification.

The convolution layers extracted local patterns in the image, the pooling layer reduced the feature map size, and the fully connected layer used the extracted features to predict the class of the digit. We used a simple model, but it was enough to demonstrate the main idea of using CNNs for MNIST digit recognition

References

Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. Chapter 9: Convolutional Networks. MIT Press. Available at: <https://www.deeplearningbook.org/contents/convnets.html>

Hanada, M. (2026). *QMUL MTH6161/MTH767P Neural Networks and Deep Learning Lecture Notes*. Queen Mary University of London.

He, K., Zhang, X., Ren, S. and Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. Available at: <https://arxiv.org/abs/1502.01852>

Stanford University. (2025). *CS231n: Convolutional Neural Networks for Visual Recognition*. Available at: <https://cs231n.github.io/convolutional-networks/>

UCI Machine Learning Repository. (2021). *MNIST Database of Handwritten Digits*. Available at: <https://archive.ics.uci.edu/dataset/683/mnist%2Bdatabase%2Bof%2Bhandwritten%2Bdigits>